

Image to Image Translation

Pix2Pix GAN

Contents

- Introduction to Pix2Pix GAN
- Difference between standard Conditional GAN and Pix2Pix.
- Applications of Pix2Pix
- U-Net Architecture for Generator Model
- Patch GAN for discriminator Model
- Weight updation of Generator and Discriminator
- Training and inference of Pix2Pix GAN

What is Pix2Pix GAN

- ✓ Pix2Pix is a specialized implementation of cGAN for image-to-image translation.
- ✓ Introduced in the paper *Image-to-Image Translation with Conditional Adversarial Networks*

Core Idea

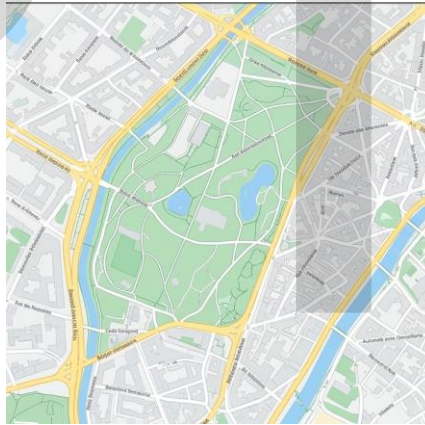
EDGE INPUT



PHOTO OUTPUT



MAP INPUT



SATELLITE OUTPUT

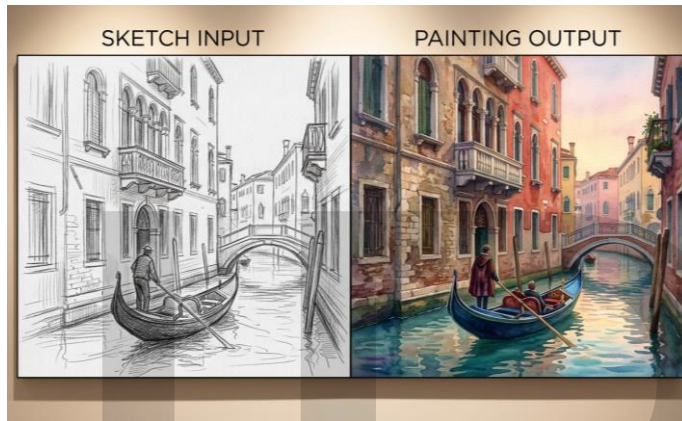


Ref:

Image-to-Image Translation with Conditional Adversarial Networks

Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, Alexei A. Efros, Berkley AI Research Lab, UC Berkley, [arXiv:1611.07004 \[cs.CV\]](https://arxiv.org/abs/1611.07004)

Core Idea



Difference from cGAN

- ✓ cGAN is a generic conditional generation framework
Can use labels, text, images, etc. as condition
- ✓ Pix2Pix is itself a **Conditional GAN (cGAN)** specifically designed for **paired image-to-image translation**.

Community Usage of Pix2Pix

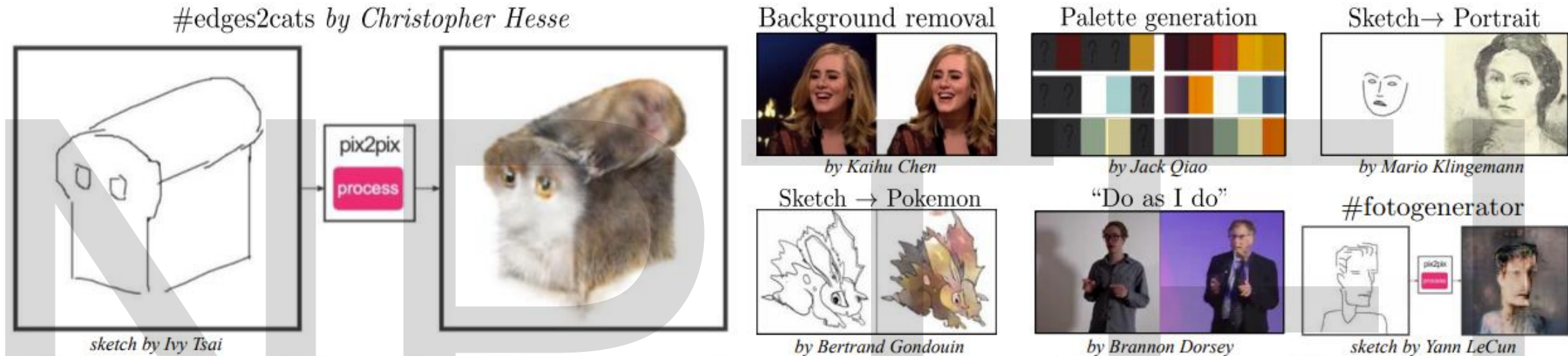
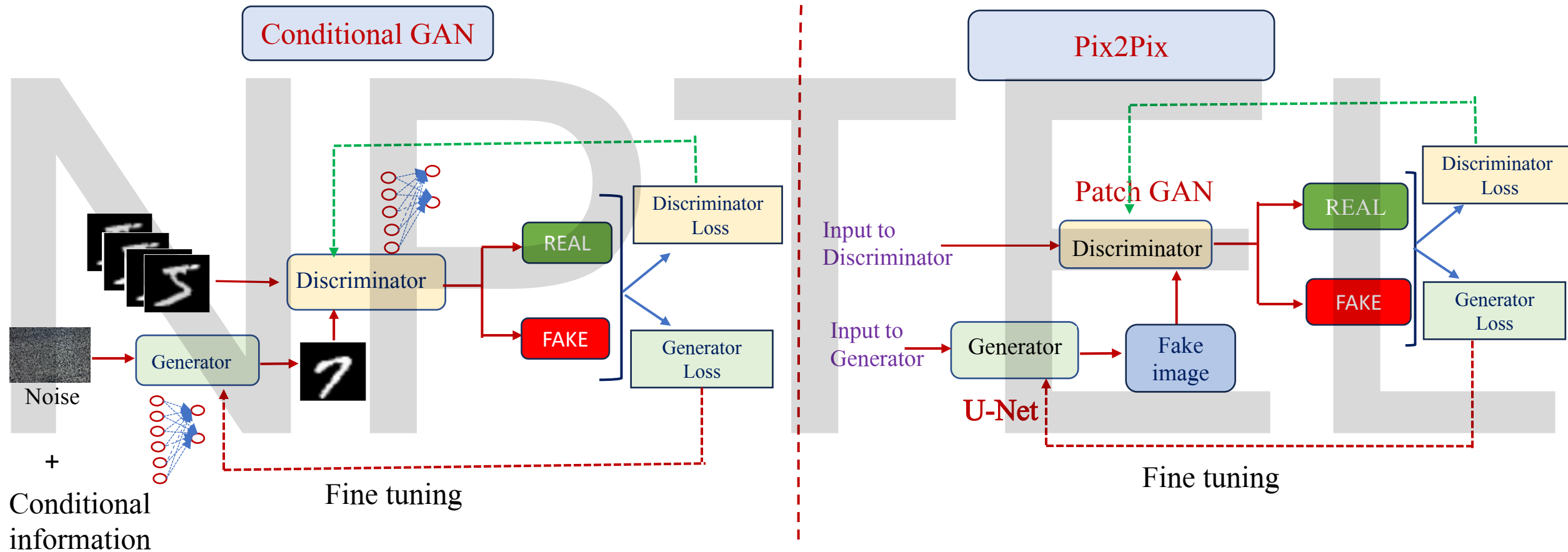


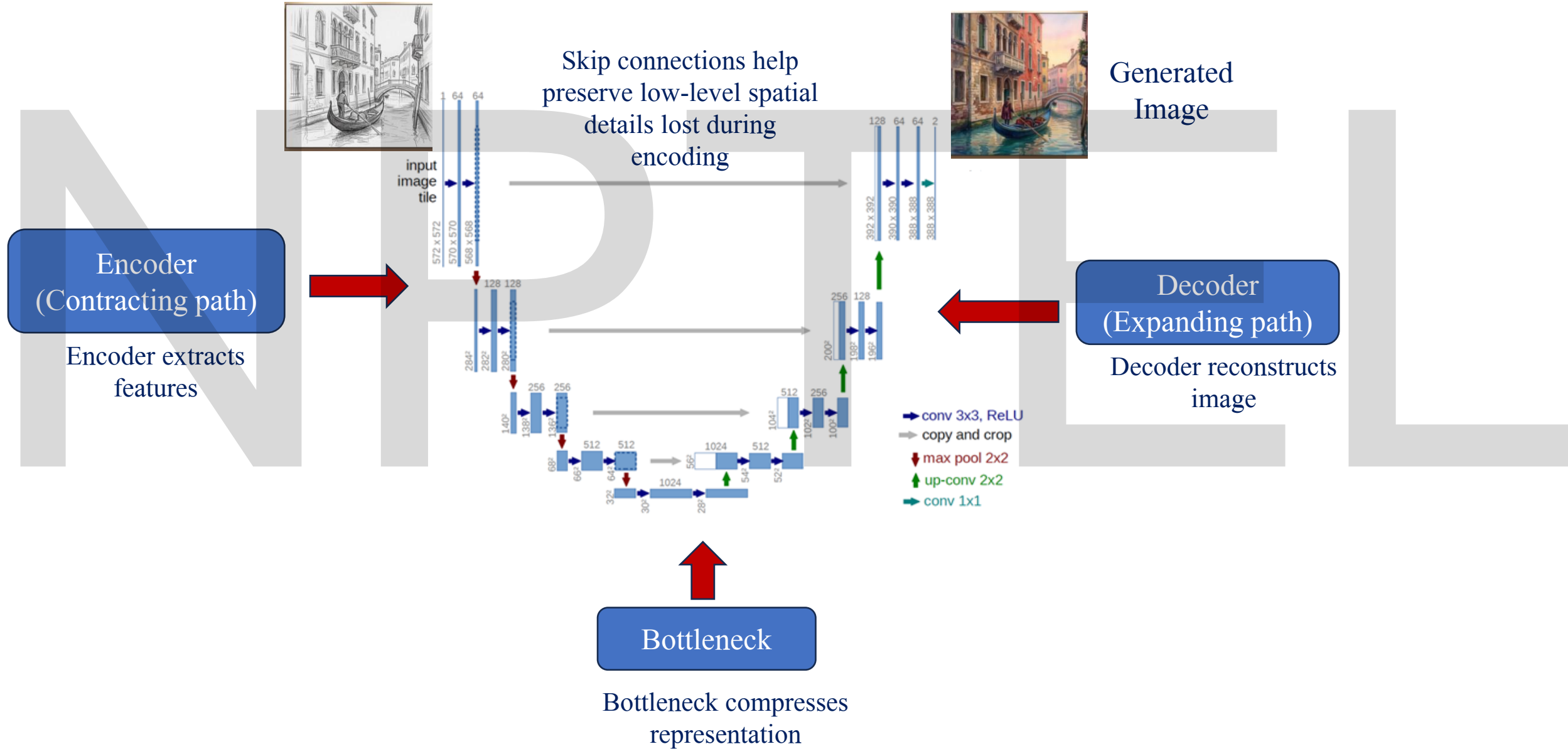
Figure 11: Example applications developed by online community based on our pix2pix codebase: *#edges2cats* [3] by Christopher Hesse, *Background removal* [6] by Kaihu Chen, *Palette generation* [5] by Jack Qiao, *Sketch → Portrait* [7] by Mario Klingemann, *Sketch → Pokemon* [1] by Bertrand Gondouin, *"Do As I Do" pose transfer* [2] by Brannon Dorsey, and *#fotogenerator* by Bosman et al. [4].

Since the initial release of the paper and our pix2pix codebase, the Twitter community, including computer vision and graphics practitioners as well as visual artists, have successfully applied our framework to a variety of novel image-to-image translation tasks, far beyond the scope of the original paper

Architectural Difference between cGAN and Pix2Pix

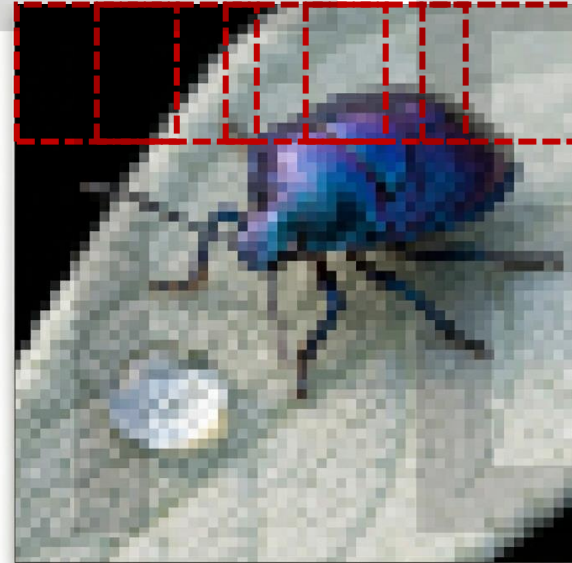


U-Net Architecture



Patch GAN as the Discriminator in Pix2Pix

A PatchGAN is a type of discriminator that classifies whether each $N \times N$ image patch is real or fake



Final discriminator output is the average of all patch decisions.

How big the Patch size be?

In the original paper, experiments have been done with different patch sizes

1×1



Only color correctness, but no effect on spatial statistics

16×16



Locally sharp results but creates tiling artifacts

70×70



Forces sharp output even if incorrect both in spatial and spectral (color) dimension

286×286



Visually similar to the 70×70 PatchGAN, Lower quality (FCN-score metric)

70×70 PatchGAN gave the best visual and FCN performance in the original paper.

Best tradeoff

Training Phase of Pix2Pix

Problem statement :

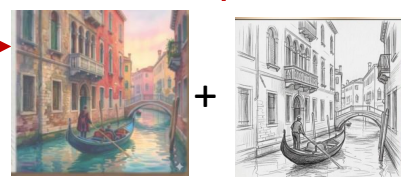
I have a sketch of a scenery (A)

I want a painting of this same scenery (B)-Target Image



Generator

U-Net



Fake B

Real A

Patch GAN

Discriminator

REAL

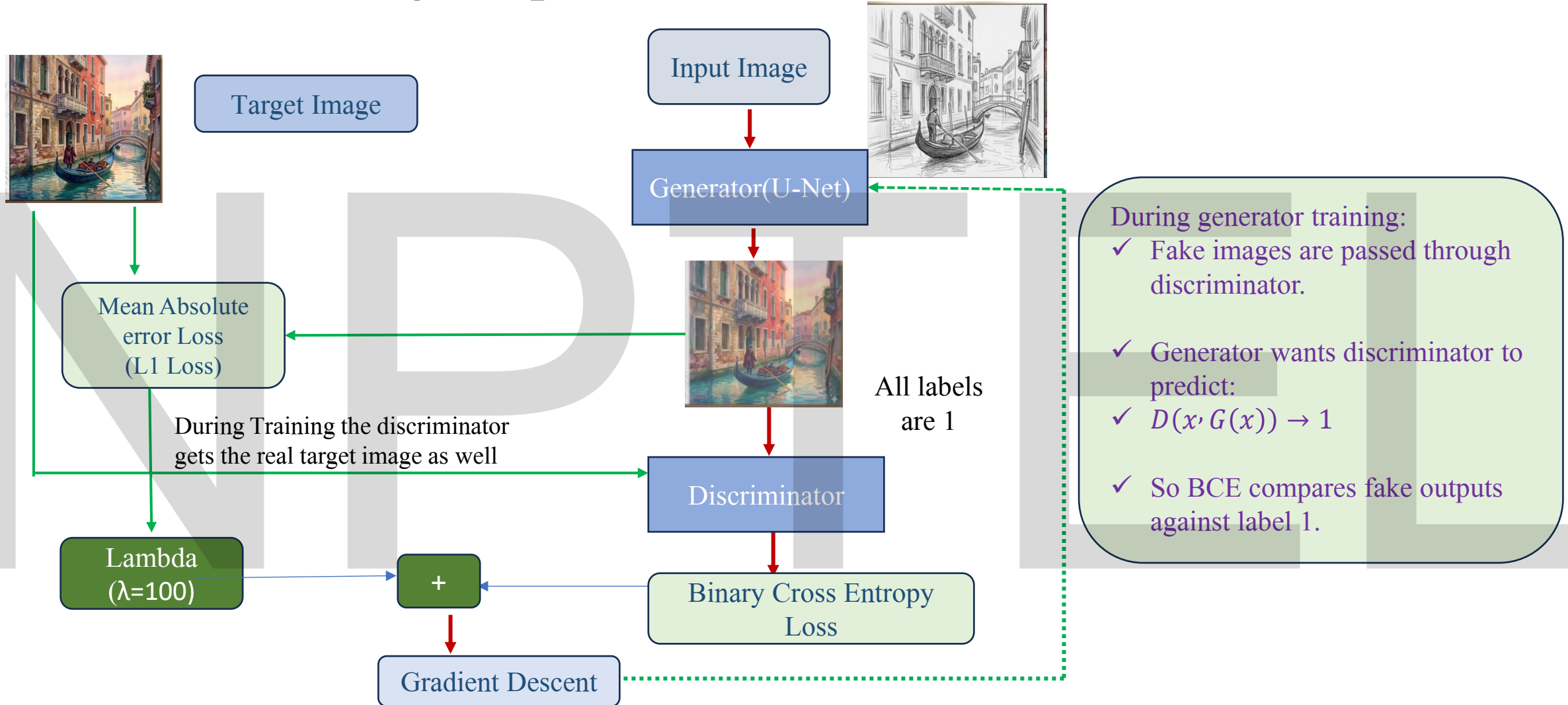
FAKE

Discriminator
Loss

Generator
Loss

GAN Loss (Binary Cross entropy loss)

Weight Updation in Generator



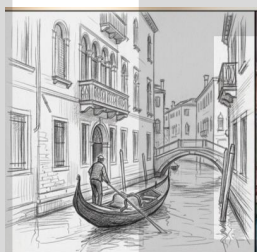
$$\text{Total Generator Loss} = \text{Binary Cross entropy Loss} + \lambda \text{ L1 loss}$$

Generator Loss of Pix2Pix GAN

$$\text{Total Loss} = \text{cGAN Loss} + \lambda \text{ L1 Loss (MAE)}$$



Adversarial Loss/GAN Loss(Conditional)



Real original
Image(x)



Real Target
Image (y)



Generated Target
Image
G(x)

In the original paper, $\lambda = 100$ was used to strongly emphasize L1 reconstruction loss

Gives strong weight to pixel similarity.

$$\mathcal{L}_{\text{cGAN}}(G, D) = \mathbb{E}_{x, y} [\log D(x, y)] + \mathbb{E}_{x, y} [\log(1 - D(x, G(x)))]$$

$$\mathcal{L}_{L1}(G) = \mathbb{E}_{x, y} [\|y - G(x)\|_1]$$

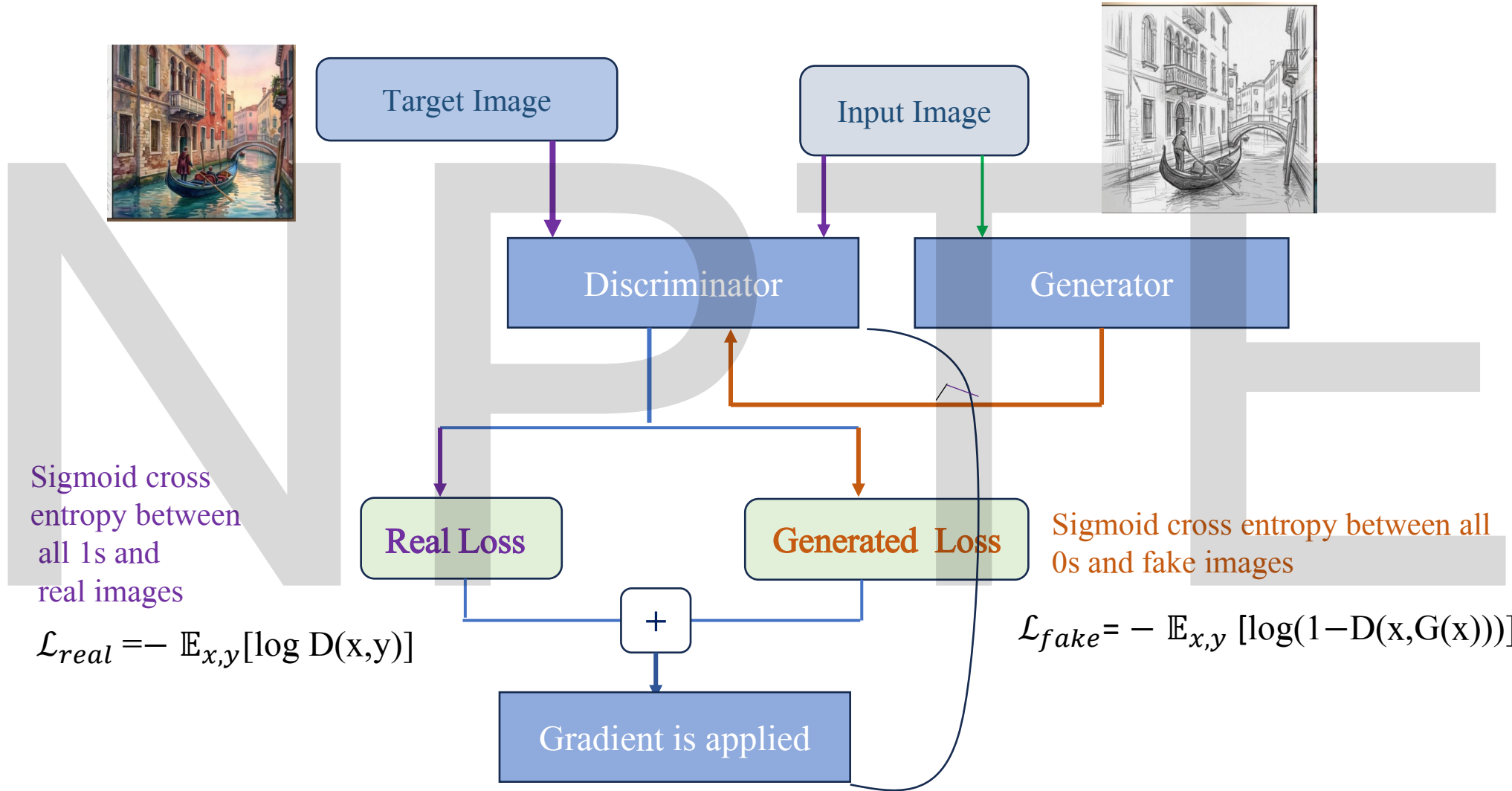
ensures the generated image is pixel-wise close to the target.

Results from the original Paper demonstrating the output quality for various types of Losses



Figure 4: Different losses induce different quality of results. Each column shows results trained under a different loss. Please see <https://phillipi.github.io/pix2pix/> for additional examples.

Weight Updation in Discriminator



$$\text{Total Discriminator Loss} = \mathcal{L}_{real} + \mathcal{L}_{fake}$$

Training Details

During training, one gradient descent step in Discriminator, followed by the generator training in an iterative way.

Optimizer:

- Adam Solver
- Learning rate: 0.0002
- $\beta_1=0.5$ and $\beta_2=0.999$

- ✓ During inference: Discriminator is not used.
- ✓ Batch size: 1-10

Objective Function of Pix2Pix

$$\mathcal{L}_{cGAN}(G, D) = \mathbb{E}_{x,y} [\log D(x, y)] + \mathbb{E}_{x,y} [\log(1 - D(x, G(x)))]$$

$$G^* = \underset{G}{\operatorname{argmin}} \max_D \mathcal{L}_{cGAN}(G, D) + \lambda \mathcal{L}_{L1}(G)$$

Generator (G) tries to minimize it

Discriminator (D) tries to maximize the objective

NPTEL

Summary

NPTEL

Next Session

Cycle GAN